CSC 489 Capstone Project: EagleMart Web Application

University of Southern Mississippi

Team Cipher

Toby Holekamp - Backend Developer

Chetanchal Saud - Frontend Developer

Benjamin Cooper - Security Tester

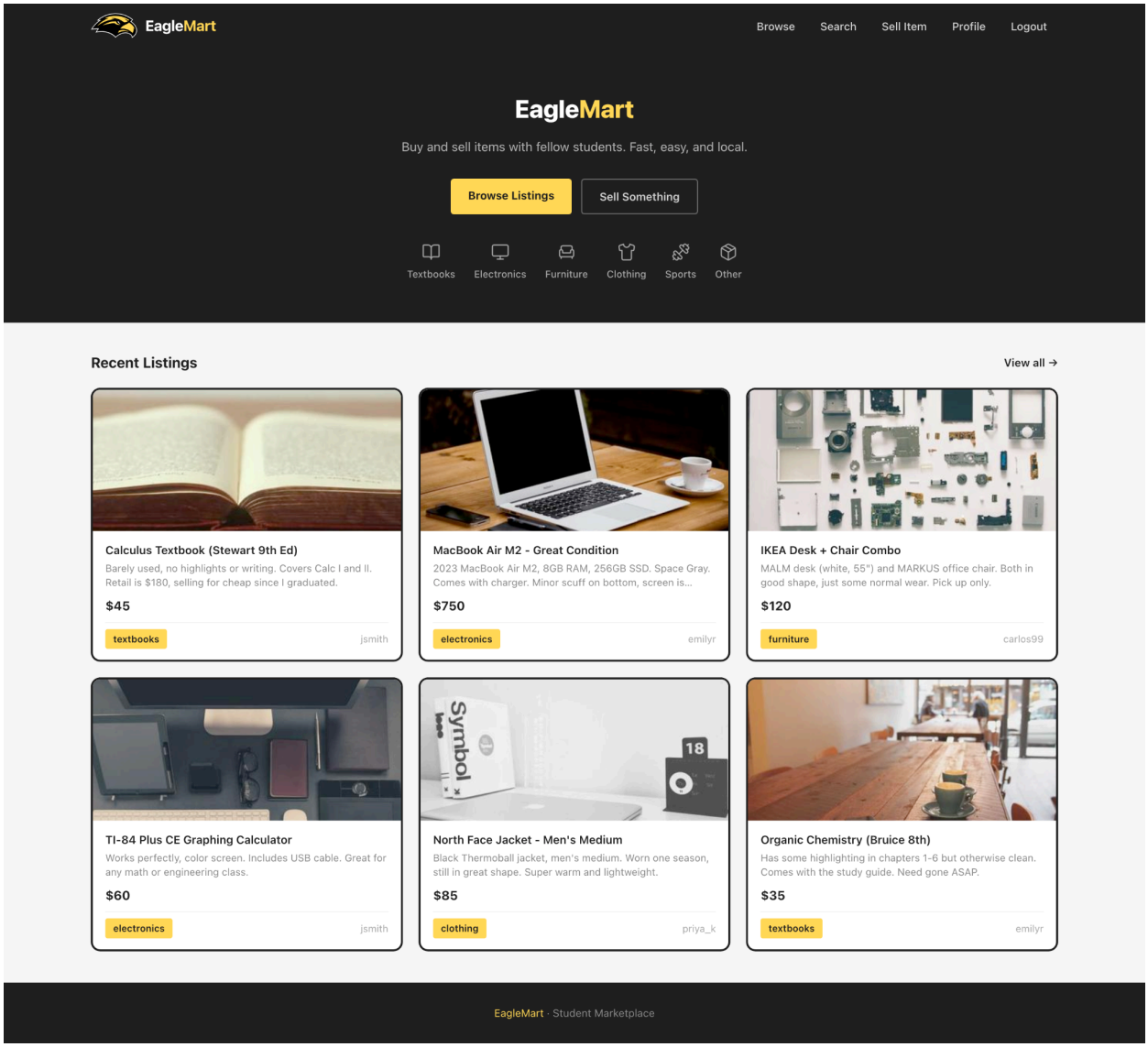
Christian Stuart - Security Tester

Binita Dhakal - Database Designer

March 29, 2026

1. Overview

EagleMart is a full-stack web application developed as a campus marketplace where students can buy and sell items such as textbooks, electronics, furniture, and other goods. The application includes key features such as user authentication, listings, search functionality, messaging, and user profiles. The main goal of this project was to develop a functional application while intentionally including common security vulnerabilities. A total of 9 vulnerabilities were implemented to reflect realistic issues that can occur in web applications .

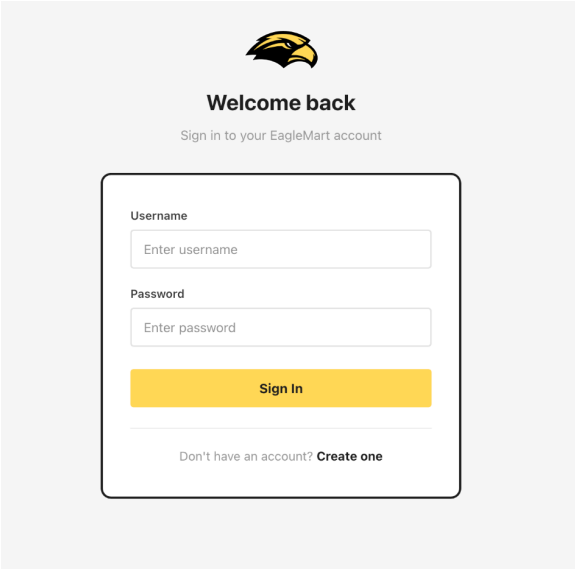


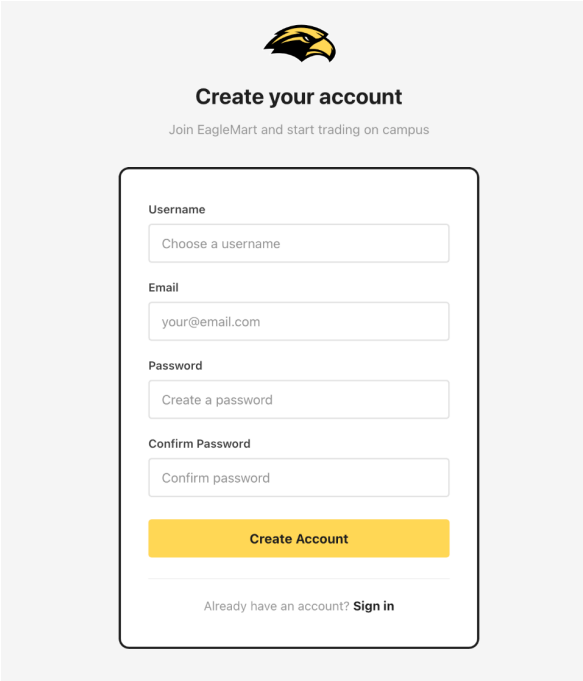
2. Features Implemented

2.1 User Authentication

Users can create accounts and log in using a username and password. The system checks credentials through the backend and stores session data on the client side.

- Frontend: /login, /register
- Backend: POST /api/login, POST /api/register

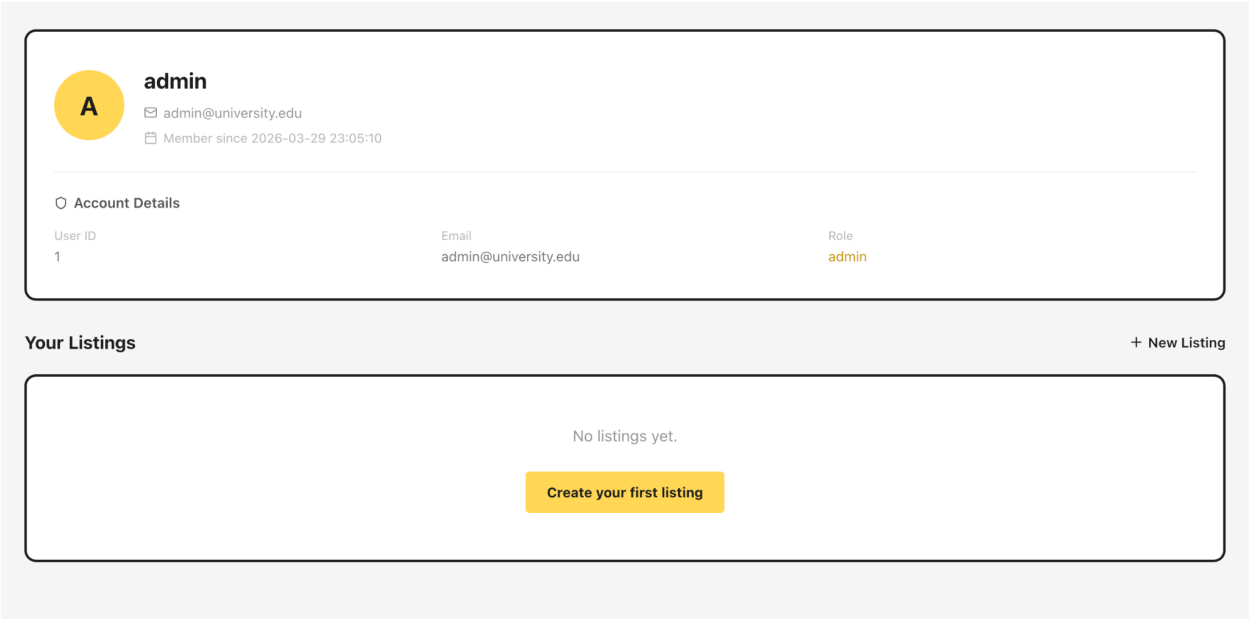




2.2 User Profiles

Each user has a profile page that displays their username, email, join date, and listings. Profiles are accessed using a numeric ID.

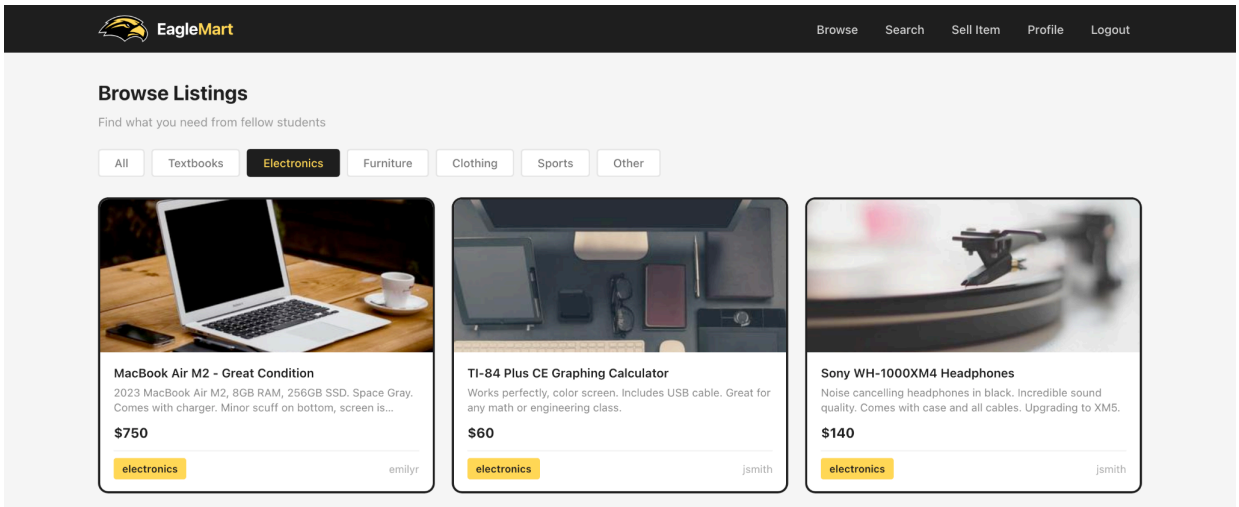
- Frontend: /profile/:id
- Backend: GET /api/profile/:id

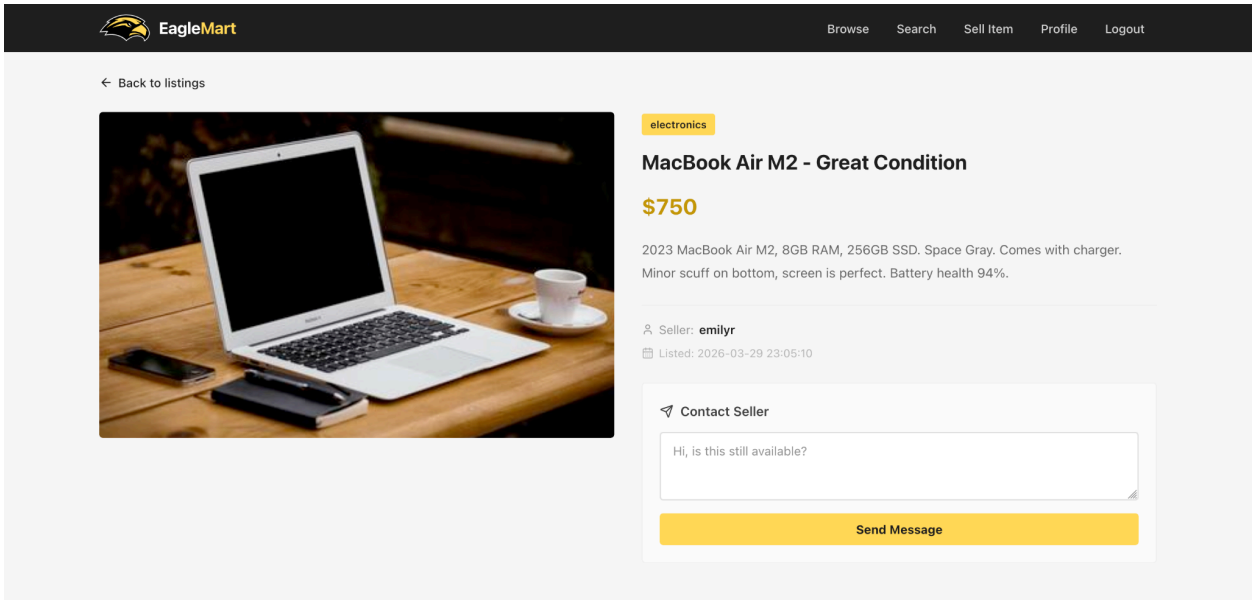


2.3 Marketplace Listings

Users can browse listings and view detailed information about each item, including description, price, and seller details.

- Frontend: /listings, /listings/:id
- Backend: GET /api/listings, GET /api/listings/:id

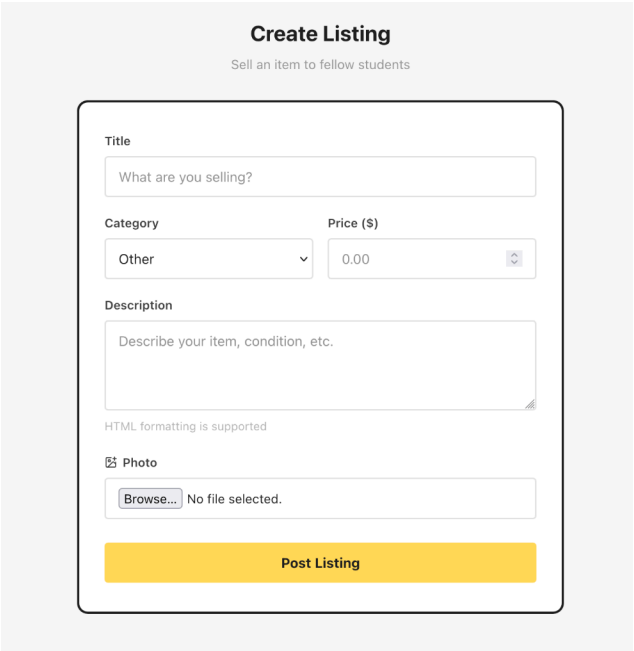




2.4 Create Listings with Image Upload

Users can create new listings by entering item details and uploading an image.

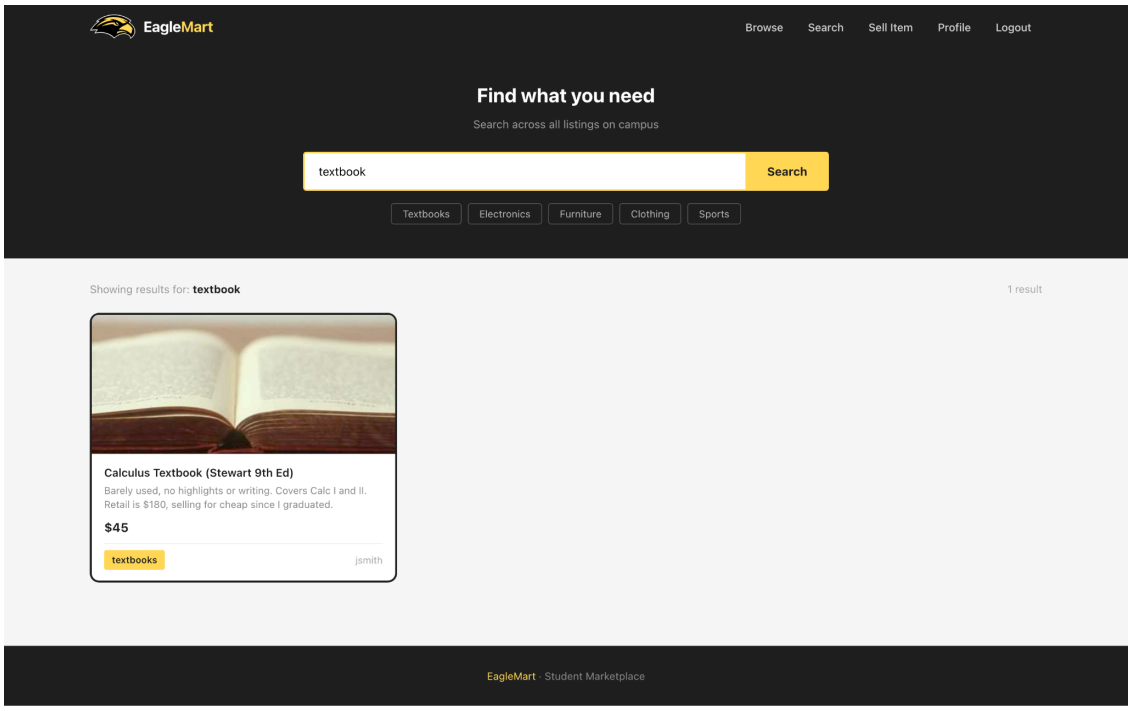
- Frontend: /create-listing
- Backend: POST /api/listings



2.5 Search & Filter

The application allows users to search listings using keywords. Category filters are also available to narrow results.

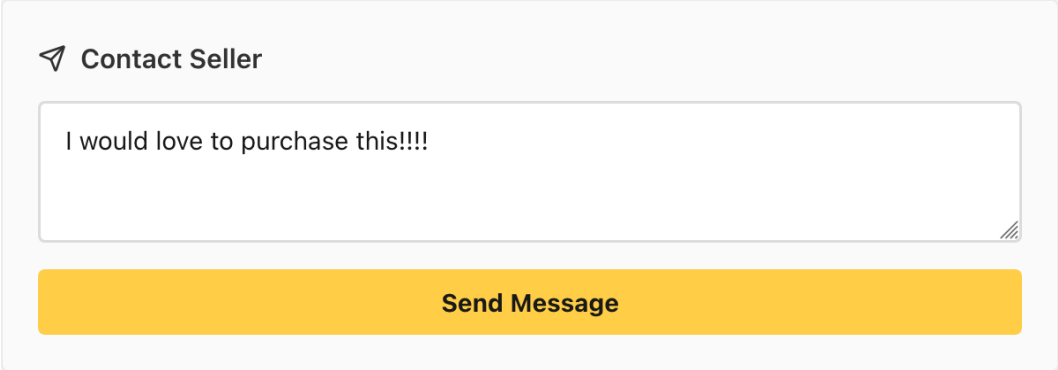
- Frontend: /search
- Backend: GET /api/search?q=

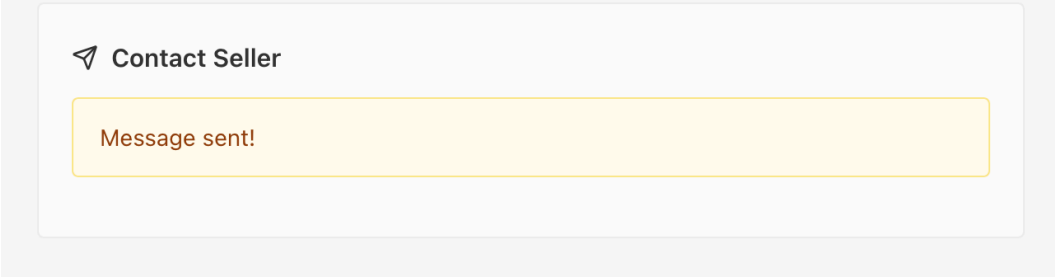


2.6 Contact Seller

Users can send messages to sellers from the listing page. Messages are stored and linked to both users and listings.

- Frontend: /listings/:id
- Backend: POST /api/messages





3. Technology Stack

The application runs locally using a startup script for both frontend and backend.

- Frontend: Next.js, React
- Backend: Flask (Python)
- Database: SQLite
- Runtime: Node.js, Python
- Libraries: Lucide React (for icons)

4. Vulnerability Documentation

4.1 SQL Injection

- **Category:** OWASP A03:2021 – Injection
- **Location:** database/db.py (line 191)
- **Description:** The search functionality is vulnerable to SQL injection because user input is directly included in SQL queries without proper validation or parameterization.
- **Impact:** An attacker can manipulate the query to bypass filters, retrieve all records, or extract sensitive data such as usernames and passwords from the database. This can lead to full database compromise.
- **Example Payload:** ' OR '1'='1

4.2 Cross-Site Scripting (XSS)

- **Category:** OWASP A03:2021 – Injection
- **Location:** frontend/app/search/page.jsx (line 142)
- **Description:** The application renders user input without proper sanitization using dangerouslySetInnerHTML, allowing malicious scripts to be injected into pages.
- **Impact:** Attackers can execute JavaScript in other users' browsers, which can be used to steal session data, redirect users, or perform actions on their behalf.
- **Example Payload:** <img src=x onerror="alert(document.cookie)">

4.3 Insecure Direct Object Reference (IDOR)

- **Category:** OWASP A01:2021 – Broken Access Control
- **Location:** backend/app.py (lines 104–119) – /api/profile/<id> endpoint
- **Description:** User profiles are accessed using a numeric ID without any authorization checks to verify if the requester has permission to view that data.
- **Impact:** An attacker can access or enumerate other users' profiles by modifying the ID in the request, leading to exposure of sensitive information such as emails and passwords.
- **Example:** Access /api/profile/1 to view admin data

4.4 Cross-Site Request Forgery (CSRF)

- **Category:** OWASP A01:2021 – Broken Access Control
- **Location:** All POST endpoints
- **Description:** The application does not use CSRF protection mechanisms such as tokens in forms or API requests.
- **Impact:** An attacker can trick a logged-in user into performing actions without their knowledge, such as creating listings or sending messages.
- **Example:** A malicious website can send a request to /api/listings while the user is logged in, creating a listing without their permission.

4.5 Insecure File Upload

- **Category:** OWASP A04:2021 – Insecure Design
- **Location:** backend/app.py (lines 150–191) – POST /api/listings
- **Description:** The file upload feature does not validate file type, size, or content, and allows files to be stored and accessed directly.
- **Impact:** Attackers can upload malicious files, including scripts, which may execute in the browser or be used to carry out further attacks on the system.
- **Example:** Upload malicious.html and access it via /uploads/malicious.html to execute script

4.6 Insecure Session Management

- **Category:** OWASP A07:2021 – Identification and Authentication Failures
- **Location:** backend/app.py (line 73) – token generation; frontend/app/components/Navbar.jsx (lines 11–17)
- **Description:** Session tokens are predictable, stored in localStorage, and not properly validated on the server.
- **Impact:** Attackers can guess or steal session tokens, allowing them to impersonate users and gain unauthorized access to accounts.
- **Example:** Set token manually to impersonate admin

4.7 Information Disclosure

- **Category:** OWASP A05:2021 – Security Misconfiguration
- **Location:** frontend/app/login/page.jsx (line 128); backend/app.py (lines 69–71)
- **Description:** The application exposes sensitive information through debug messages, error responses, and public user data.
- **Impact:** Attackers can gather useful details such as valid usernames, system structure, and credentials, which can help in further attacks.
- **Example:** View page source to find credentials in HTML comments

4.8 Broken Authentication

- **Category:** OWASP A07:2021 – Identification and Authentication Failures
- **Location:** backend/app.py – POST /api/messages (lines 216–237), POST /api/listings (lines 148–185)
- **Description:** Some API endpoints do not enforce authentication and trust user-provided data without verification.
- **Impact:** Attackers can perform actions such as creating listings or sending messages without being logged in, and can impersonate other users by modifying request data.
- **Example:** Send a POST request with seller_id=1 to create a listing as admin

4.9 Sensitive Data Exposure

- **Category:** OWASP A02:2021 – Cryptographic Failures
- **Location:** database/db.py – password storage; backend/app.py (line 73) – password returned in responses
- **Description:** User passwords are stored and returned in plain text without encryption or hashing.
- **Impact:** If accessed, all user credentials can be easily read and misused, leading to account compromise and potential system-wide access.
- **Example:** Access /api/profile/2 to retrieve plaintext password

5. Development Notes

The EagleMart application was designed to simulate a real-world web system while keeping the structure simple and easy to test. The backend was implemented using a single Flask file to make all API routes easy to review, and SQLite was used as the database to avoid complex setup. The frontend was built with Next.js to provide a modern and responsive interface.

The vulnerabilities were intentionally included in areas that reflect common developer mistakes, such as improper input handling, missing validation, weak authentication checks, and insecure data storage. The application was tested in a local environment, and all core features function as expected. The system runs without major errors and supports all implemented functionality. It is intended only for testing and learning purposes, and default credentials are included to simplify evaluation. Some features, such as messaging, are basic and do not include a full user interface. The goal was to demonstrate how small oversights can lead to serious security issues in a real application.